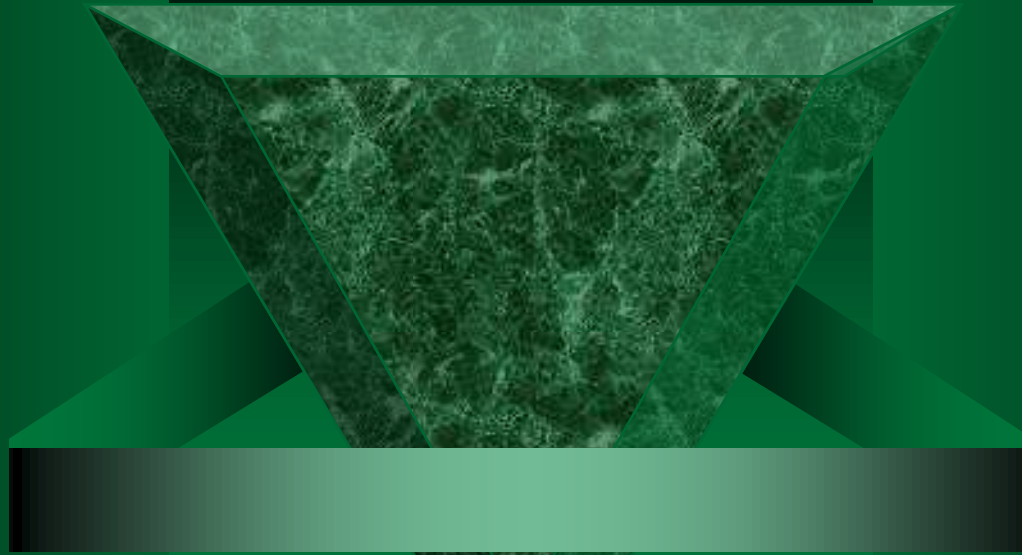


Apirak :



6.2 Recursive Function



- t Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.

program organization on c/c++ :

```
#include <stdio.h>

int addNumbers(int a, int b);

int main()
{
    ... ..
    sum = addNumbers(n1, n2);
    ... ..
}

int addNumbers(int a, int b)
{
    ... ..
    return result;
}
```

Function Parameters

Function Prototype

Main Function

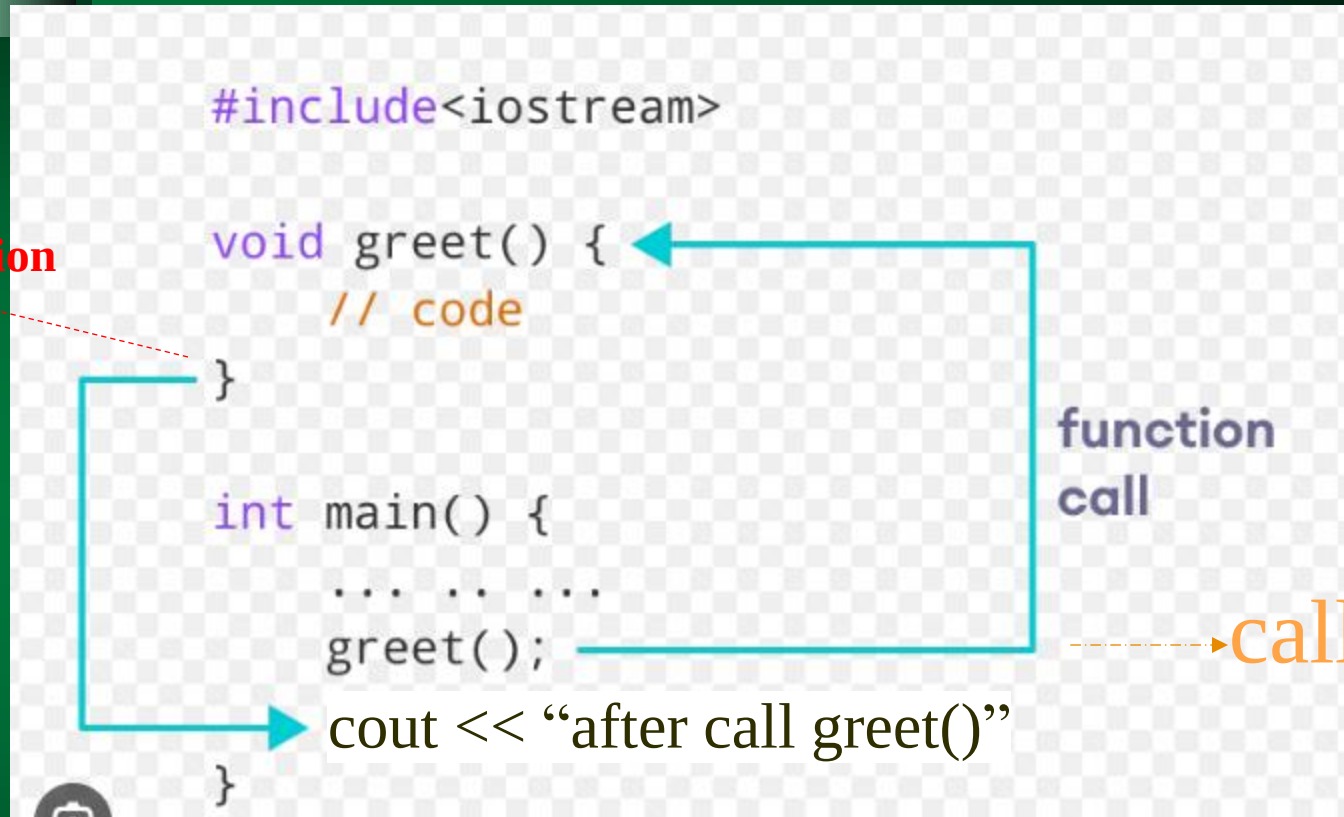
Function Call Statement

Function Declaration

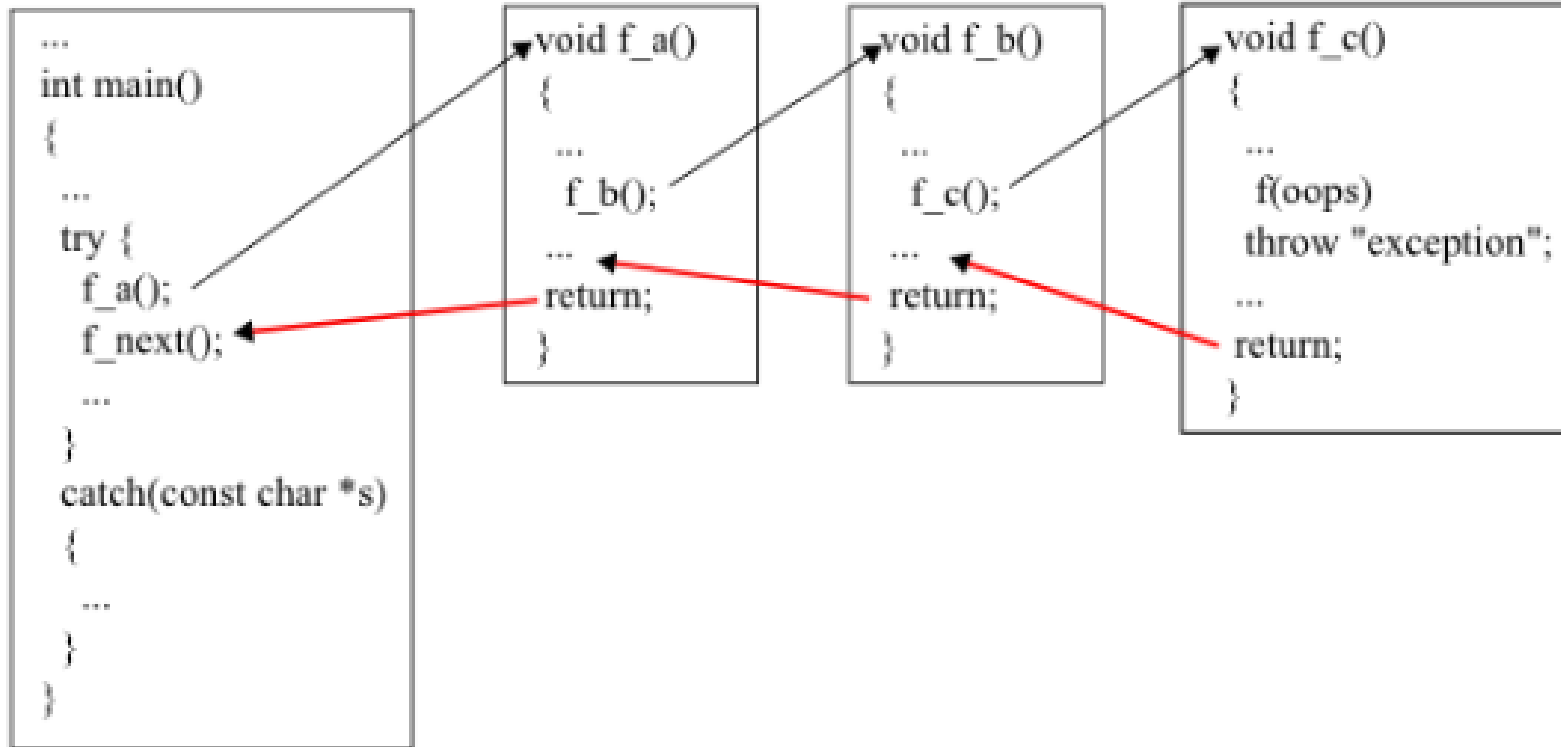
Function Arguments

Execution flows (1)

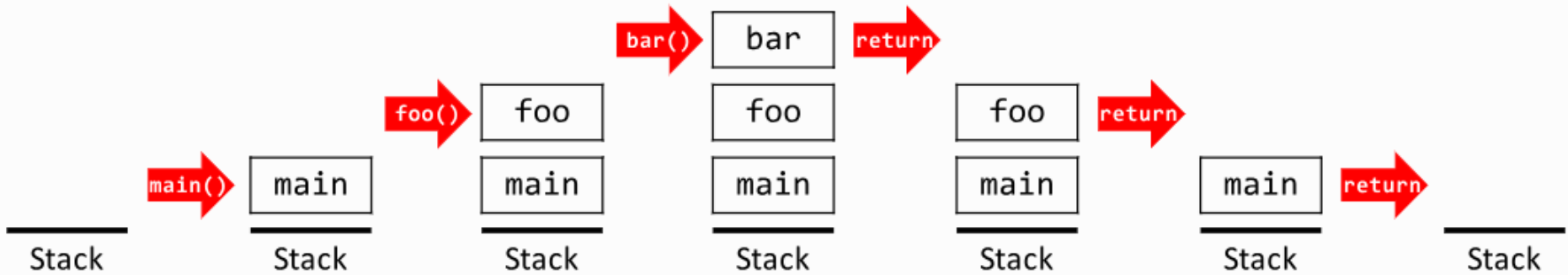
End fuction



Execution flows (2)



Execution flows (3)



Operating System

call

```
void bar() {  
}  
  
void foo() {  
    bar();  
}  
  
int main() {  
    foo();  
}
```

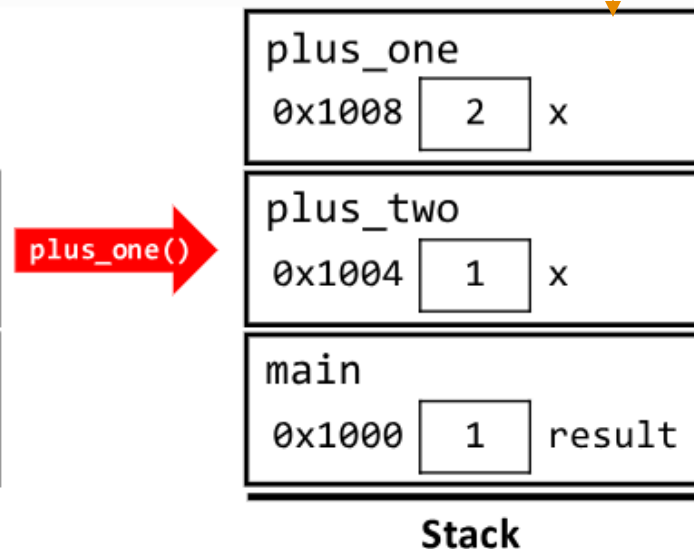
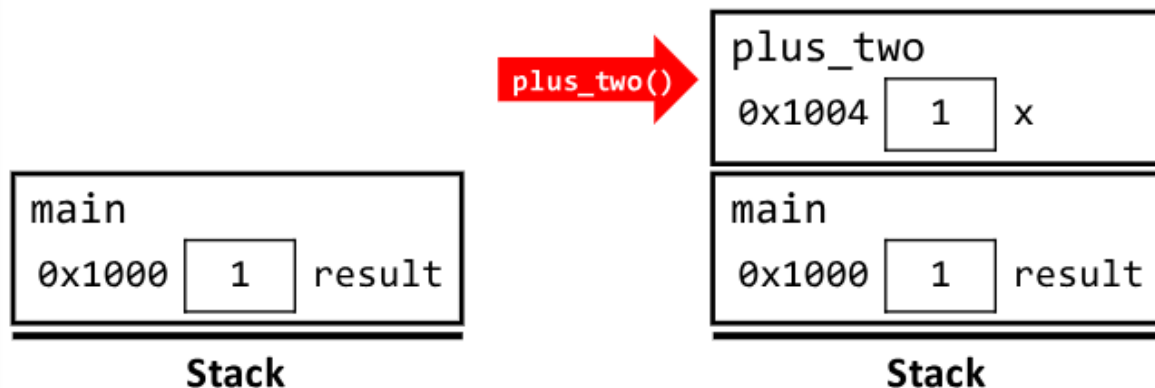
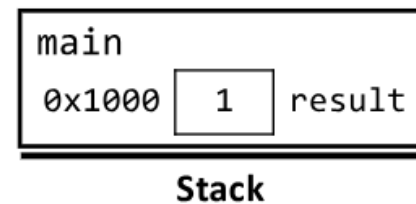
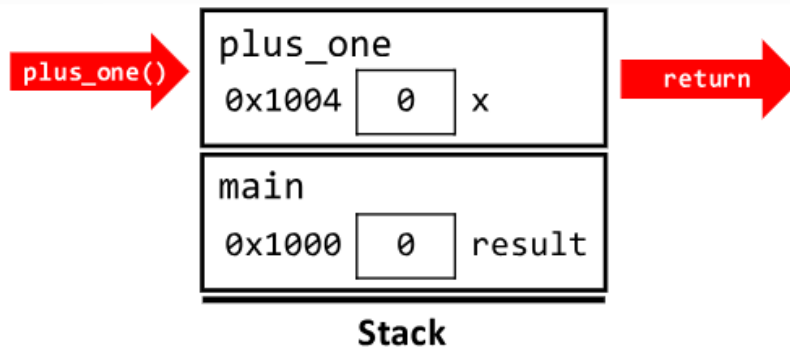
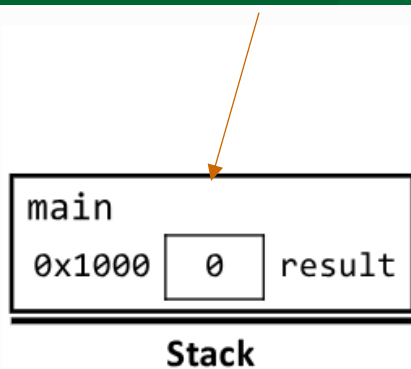
Execution flows (4)

OS call

```
int plus_one(int x) {  
    return x + 1;  
}
```

```
int plus_two(int x) {  
    return plus_one(x + 1);  
}
```

```
int main() {  
    int result = 0;  
    result = plus_one(0);  
    result = plus_two(result);  
    cout << result;  
} // print
```





Implementing a Recursive Function

- I. It is a **recursive** function – a function that calls itself.
- II. It will keep on calling itself, with different parameters, until a terminating condition is met.
- III. **It's like writing a loop. You can also write a loop to do something over and over again, until some exiting condition is met.**
- IV. **Resolution the recursion mathematic function problem**



Recursion

Direct Recursion

```
void f()  
{  
    . . .  
    f();  
    . . .  
}
```

(a)

Indirect Recursion

```
void a()  
{  
    . . .  
    b();  
    . . .  
}
```

```
void b()  
{  
    . . .  
    c();  
    . . .  
}
```

```
void c()  
{  
    . . .  
    d();  
    . . .  
}
```

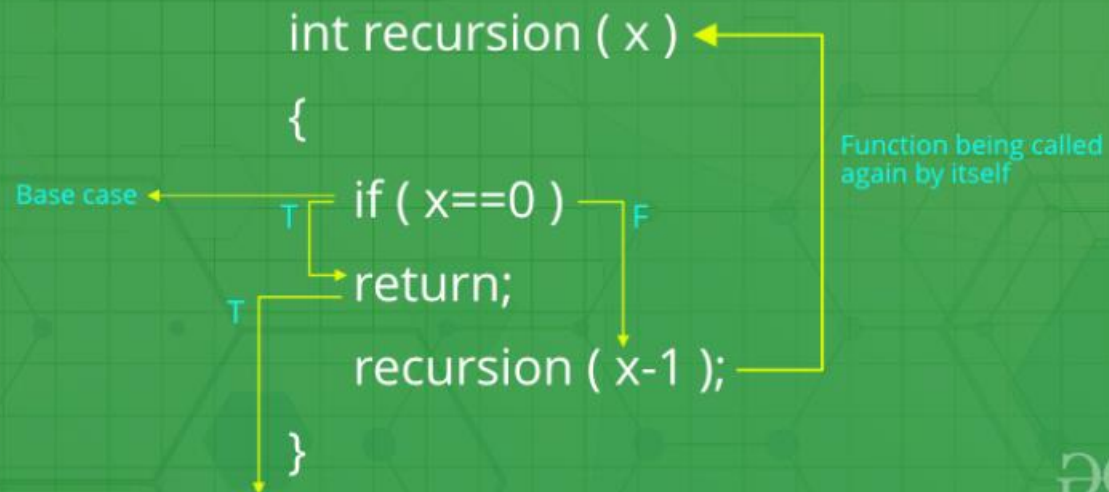
```
void d()  
{  
    . . .  
    a();  
    . . .  
}
```

(b)

Figure 1. Direct and indirect recursion.

Recursive function pattern

Recursive Functions



4. forward(0)
5. cout << 1

3. forward(1)
6. cout << 2

2. forward(12)
7. cout << 3

1. forward(123)

(a)

number: 0
number: 1
number: 12
number: 123

4th

3rd

2nd

1st

6. cout << 1
7. reverse(0)

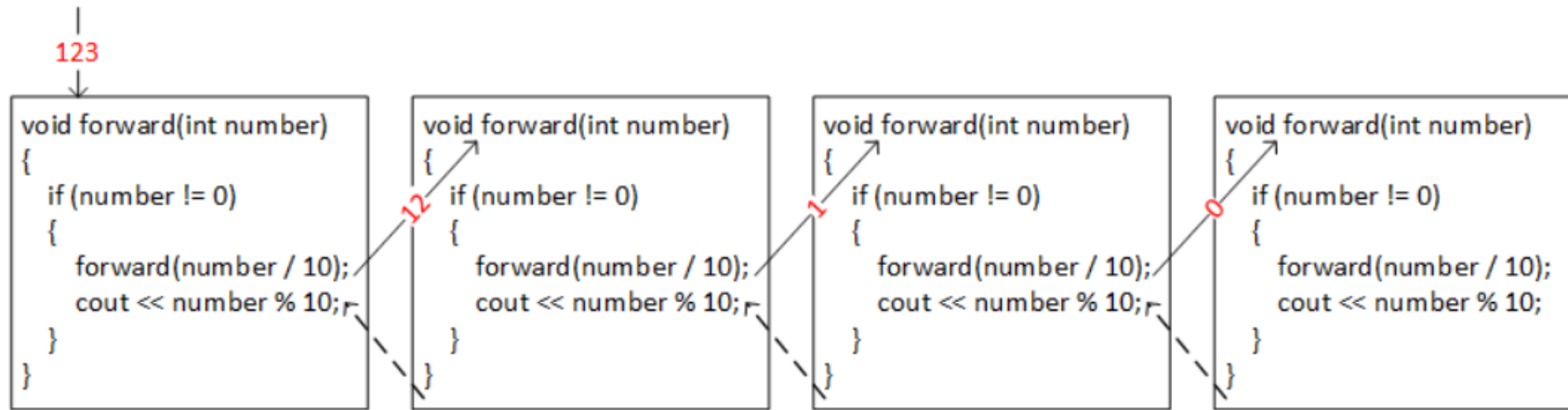
4. cout << 2
5. reverse(1)

2. cout << 3
3. reverse(12)

1. reverse(123)

(b)

number: 0
number: 1
number: 12
number: 123



(c)

```
public class Recurse1
```

```
{
```

```
    public static void main( String[] args )
```

```
    {
```

```
        double r;
```

```
        r = factorial( 3 ); ← point A
```

```
    }
```

```
    public static int factorial ( int n )
```

```
    {
```

```
        int sol1, solution;
```

```
        if ( n == 0 )
```

```
        {
```

```
            return 1;
```

```
        }
```

```
        else
```

```
        {
```

```
            sol1 = factorial(n-1); ← point B
```

```
            solution = n * sol1;
```

```
            return( solution );
```

```
        }
```

```
    }
```

```
}
```

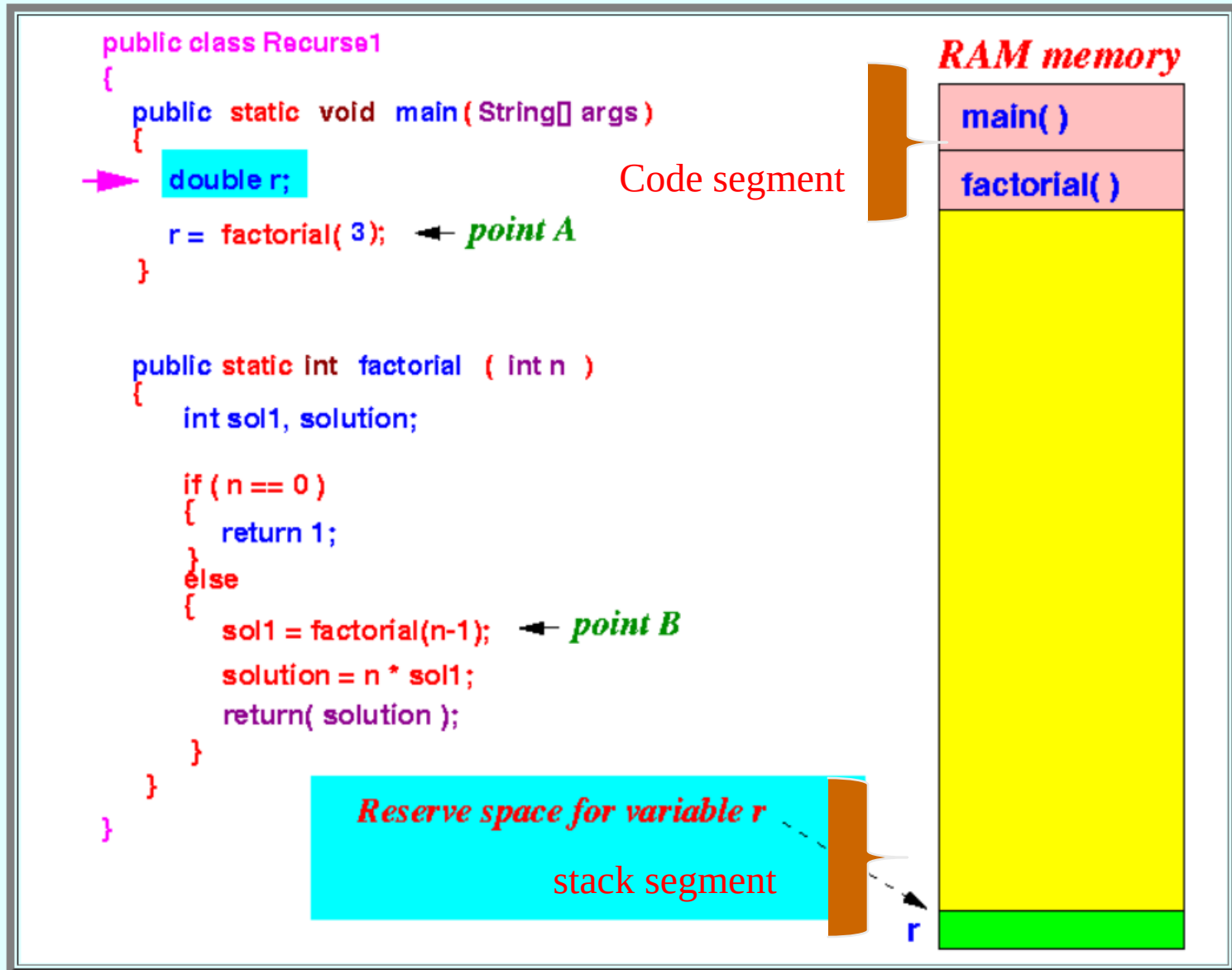
Code segment

RAM memory

main()

factorial()

- Execution starts in `main()`:



Reserves space for local variable `r` of `main()`

- main() calls factorial(3) - passes parameter 3

```
public class Recurse1
{
    public static void main (String[] args)
    {
        double r;
        r = factorial( 3);
    }
```

← point A

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1);
        solution = n * sol1;
        return( solution );
    }
}
```

← point B

return
n
r

RAM memory

main()

factorial()

point A

3

- `factorial(3)` calls `factorial(n-1 (2))` - passes parameter 2

```
public class Recurse01
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1); ← point B
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory

main()

factorial()

return
n
solution
sol1
return
n
r

point B

2

point A

3

Notice that a *new copy* of the *parameter variable n* is *created*.

Apurak :

-

Apirak :

- `factorial(2)` calls `factorial(n-1 (1))` - passes parameter 1

```

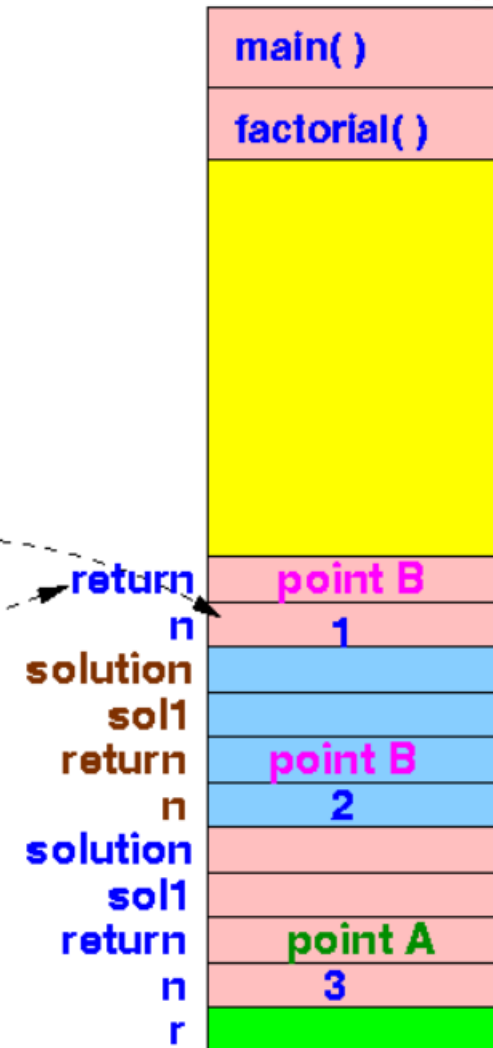
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3 ); ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            ← sol1 = factorial(n-1); ← point B
            solution = n * sol1;
            return( solution );
        }
    }
}

```

RAM memory



Notice that yet another **new copy** of the **parameter variable n** is created.

Now the **execution again goes to the factorial method** !!!

- `factorial(1)` calls `factorial(n-1 (0))` - passes parameter 0

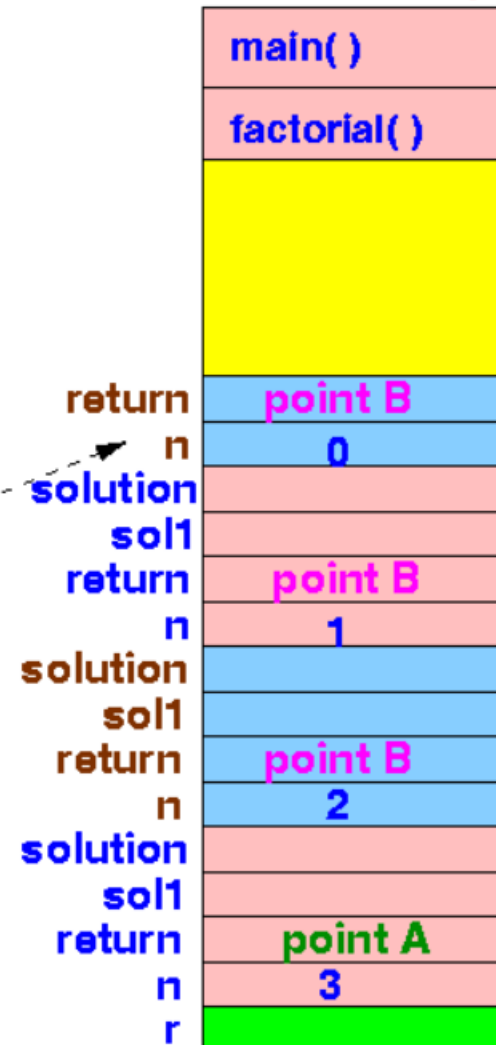
```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3); ← point A
    }
}
```

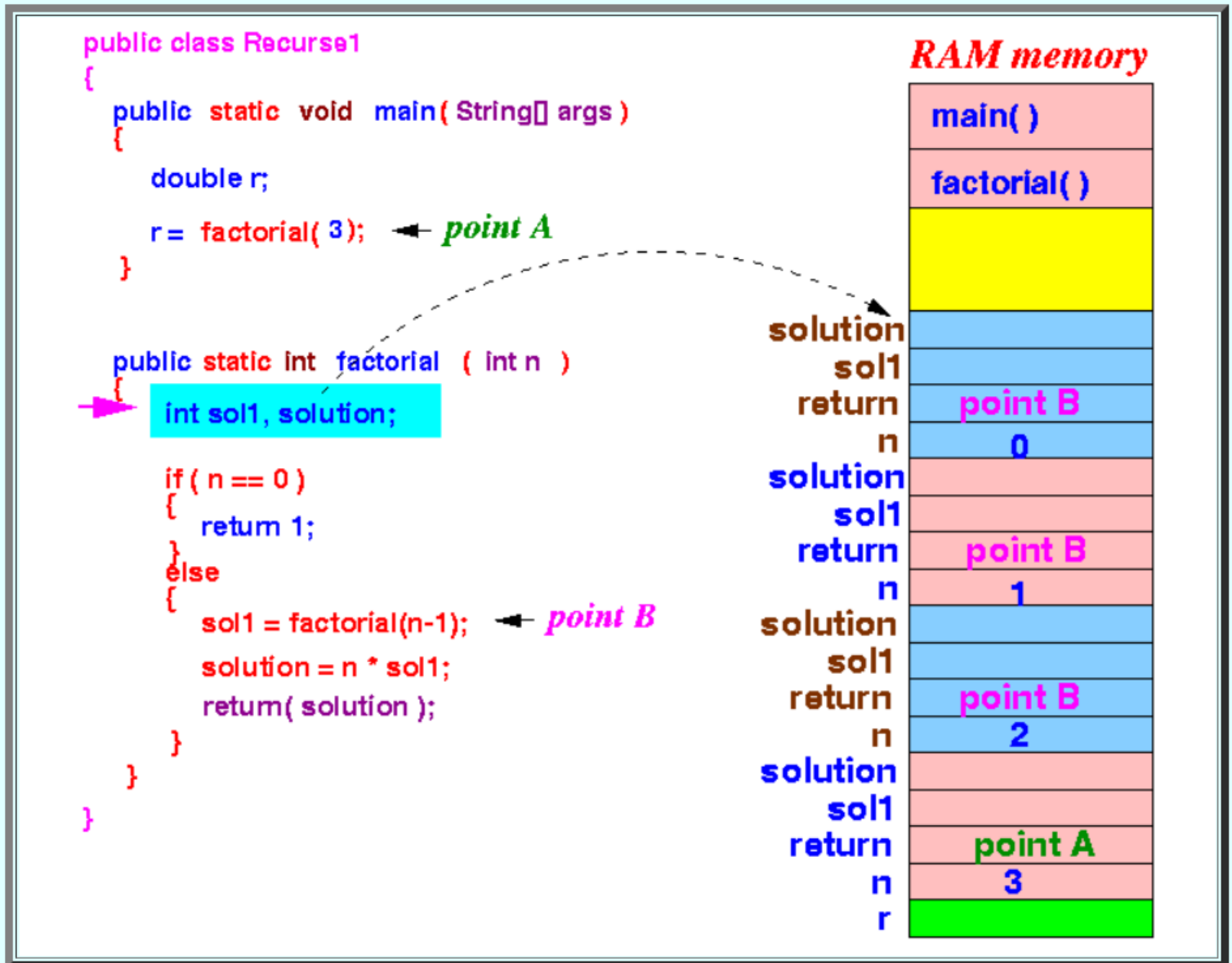
```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1); ← point B
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory



- `factorial(0)` will create the **local variables**:



- factorial(0) will *return the value 1*:

```

public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3); ← point A
    }

    public static int factorial ( Int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            sol1 = factorial(n-1); ← point B
            solution = n * sol1;
            return( solution );
        }
    }
}

```

RAM memory

	main()
	factorial()
solution	
sol1	
return	point B
n	0
solution	
sol1	
return	point B
n	1
solution	
sol1	
return	point B
n	2
solution	
sol1	
return	point A
n	3
r	

- Execution returns to *point B* in `factorial(2)`:

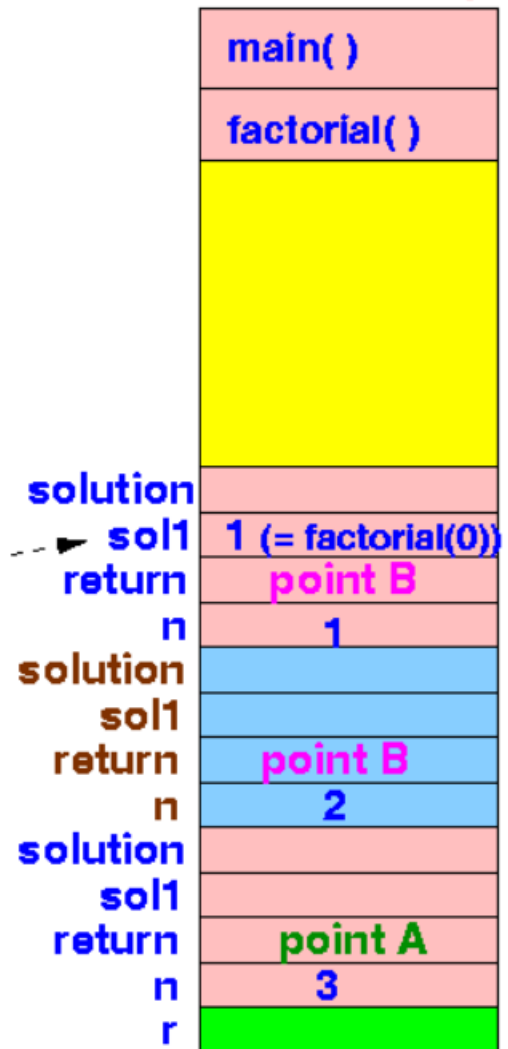
```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1); ← point B
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory



`factorial(1)` will use the solution `sol1` and return `solution = 1 × 1 = 1`

- Execution **returns** to *point B* in `factorial(2)`:

```

public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

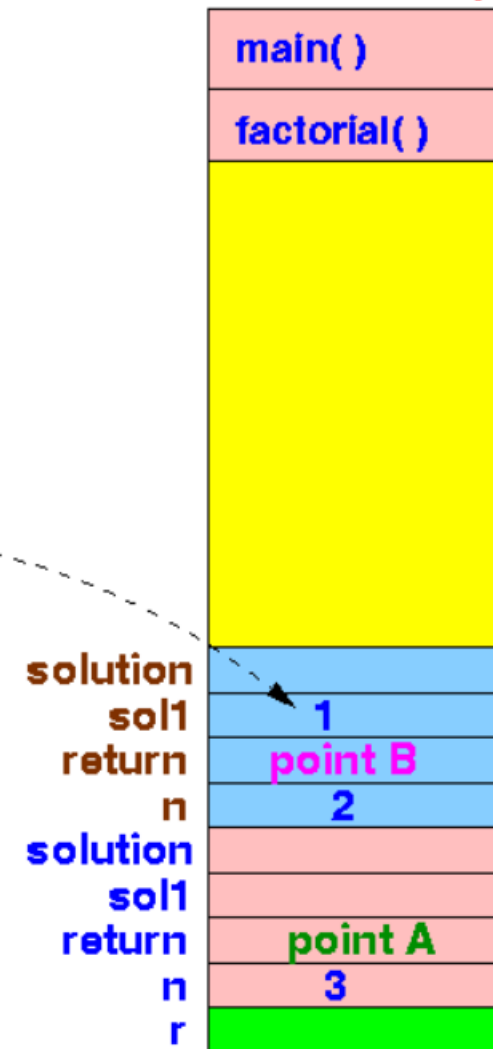
        r = factorial( 3 ); ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            sol1 = factorial(n-1); ← point B
            solution = n * sol1;
            return( solution );
        }
    }
}

```

RAM memory



`factorial(2)` will use the solution `sol1 (= 1)` and **return** `solution = 2 × 1 = 2`

- Execution returns to *point B* in `factorial(3)`:

```

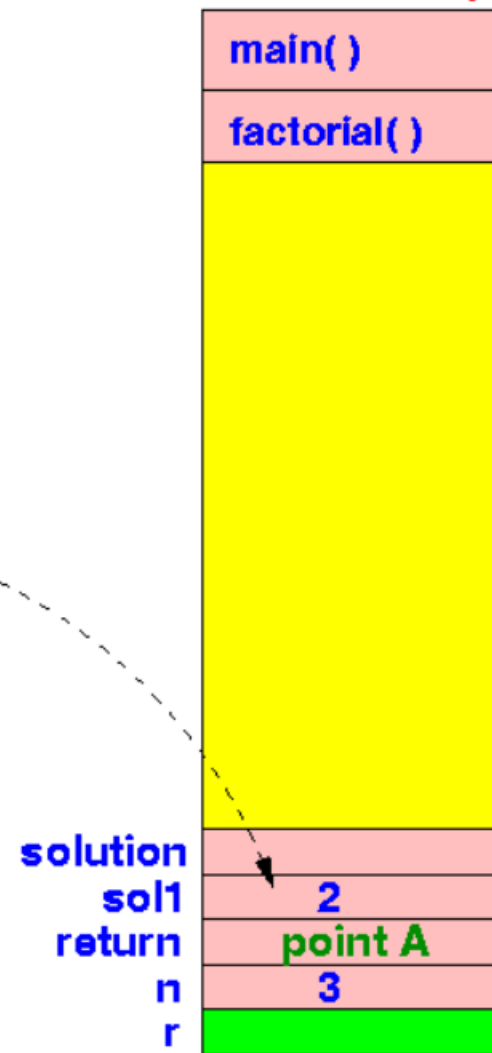
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3); ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            ← point B
            sol1 = factorial(n-1);
            solution = n * sol1;
            return( solution );
        }
    }
}

```

RAM memory



`factorial(3)` will use the solution `sol1 (= 2)` and return `solution = 3 × 2 = 6`

- Finally... the **original** call of `factorial(3)` will *return the value 26*:

```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3 ); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1);
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory

main()
factorial()
- - ➤ 6 (=3x2)
2
point A
3

`factorial(3)` will now **terminate**.

- Execution returns to *point A* in `main()`:

```
public class Recurse1
{
    public static void main (String[] args)
    {
        double r;
        r = factorial( 3);
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            sol1 = factorial(n-1);
            solution = n * sol1;
            return( solution );
        }
    }
}
```

RAM memory

```
main( )
```

factorial()

r

6

We are **finally** done !!!

Apirak :

```

1
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     for(int i=1 ; i<=3 ; i++)
8     {
9         cout << "\n Line No." << i;
10    }
11    return 0;
12 }
13
14

```

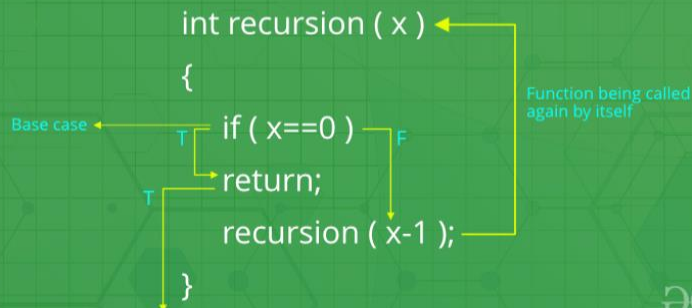
nain.cpp

```

1
2 #include <iostream>
3 using namespace std;
4
5 void f2(int i)
6 {
7     if(i<=3) {
8         cout << "\n Line No." << i;
9         f2(i+1);
10    }
11 }
12
13 int main()
14 {
15     f2(1);
16     return 0;
17 }
18

```

Recursive Functions

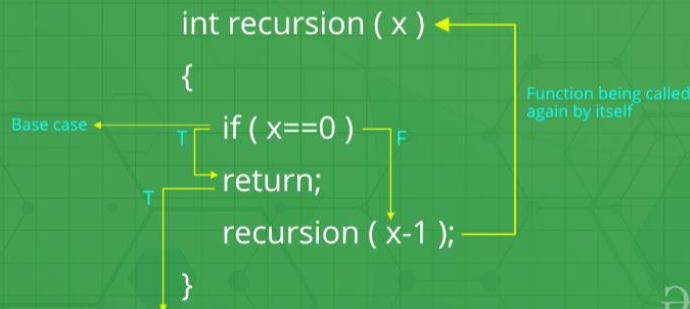


main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int s =0;
7      //s = 1+2+3
8      for(int i=1 ; i<=3 ; i++)
9      {
10         s = s+i;
11     }
12     cout << "\nsum = " << s;
13     return 0;
14 }
15
```

```
2  #include <iostream>
3  using namespace std;
4
5  int f2(int i)
6  {
7      //s = 1+2+3
8      if(i>=3)
9      {
10         return i;
11     }
12     return i+f2(i+1);
13 }
14
15 int main()
16 {
17     cout << "\nsum = " << f2(1);
18     return 0;
19 }
20
21
```

Recursive Functions



```

1
2 #include <iostream>
3 using namespace std;
4
5 int f2(int i)
6 {
7     //s = 1+2+3
8     if(i>=3)
9     {
10         return i;
11     }
12     return i+f2(i+1);
13 }
14
15 int main()
16 {
17     cout << "\nsum = " << f2(1);
18     return 0;
19 }
20
21

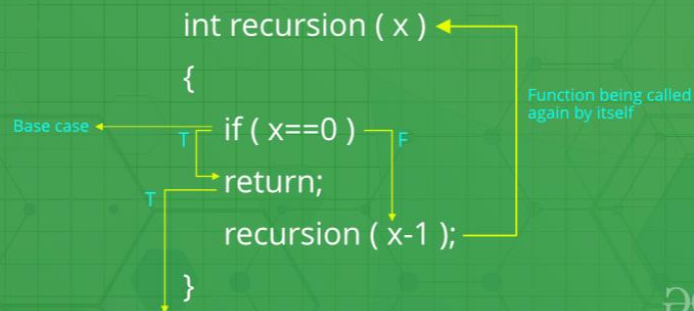
```

```

22 #include <iostream>
23 using namespace std;
24
25 int f2(int i)
26 {
27     ....//s = 4+3+2+1
28     ....if (i<=0)
29     ....{
30     ....return 0;
31     ....}
32     ....return i+f2(i-1);
33 }
34
35
36 int main()
37 {
38     ....cout << "\nsum = " << f2(4);
39     ....return 0;
40 }

```

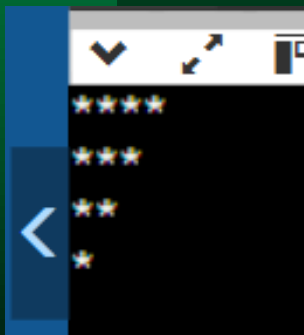
Recursive Functions



```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n = 4;
7      while(n>0)
8      {
9          for(int j = 0 ; j< n ; j++)
10         {
11             cout << "*";
12         }
13         cout << "\n";
14         n--;
15     }
16     return 0;
17 }
18

```



```

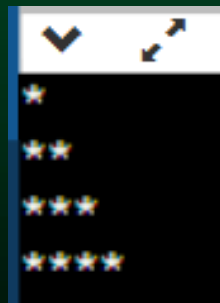
1
2  #include <iostream>
3  using namespace std;
4
5  void f2(int n)
6  {
7      if(n<=0) {
8          return ;
9      }
10     for(int j = 0 ; j< n ; j++)
11     {
12         cout << "*";
13     }
14     cout << "\n";
15     f2(n-1);
16 }
17
18 int main()
19 {
20     f2(4);
21     return 0;
22 }
23
24

```

```

1
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int n=4;
8     for(int i=1; i<=n ; i++)
9     {
10         for(int j = 0 ; j< i ; j++)
11         {
12             cout << "*";
13         }
14         cout << "\n";
15     }
16     return 0;
17 }

```



```

*
**
***
****

```

```

main.cpp
1
2 #include <iostream>
3 using namespace std;
4
5 void f1(int j,int n)
6 {
7     if(j<=n) {
8         for(int i=1; i<=j ; i++)
9         {
10             cout << "*";
11         }
12         cout << "\n";
13         f1(j+1,n);
14     }
15 }
16
17 int main()
18 {
19     f1(1,4);
20     return 0;
21 }
22
23

```



จงเขียนโปรแกรมแปลงเลขฐาน 10 เป็น 2

2 221	เศษ	1 (LSB)
2 110	เศษ	0
2 55	เศษ	1
2 27	เศษ	1
2 13	เศษ	1
2 6	เศษ	0
2 3	เศษ	1
2 1	เศษ	1
0		

$$\therefore (221)_{10} = (11011101)_2$$

```
2  #include <iostream>
3  using namespace std;
4
5  int main( )
6  {
7      int x , fac , i ;
8      cout << "pls, enter data (int) : ";
9      cin >> x ;
10     while(x !=0 )
11     {
12         int b;
13         b = x%2;
14         x = x/2;
15         cout << b << endl;
16     }
17     cout << fac << endl;
18 }
19
```

โปรแกรมแปลงฐาน 10 เป็นฐาน 2 และ 8

```
259 void dec2bin(int x, int bin[], int i, int &n)
260 {
261     if(x!=0) {
262         int b;
263         b = x%2;
264         x = x/2;
265         //cout << b;
266         bin[i] = b;
267         n = i+1;
268         dec2bin(x, bin, i+1, n);
269     }
270 }
271
272 void dec2hex(int x, int bin[], int i, int &n)
273 {
274     if(x!=0) {
275         int b;
276         b = x%16;
277         x = x/16;
278         //cout << b;
279         bin[i] = b;
280         n = i+1;
281         dec2hex(x, bin, i+1, n);
282     }
283 }
284
```



```
285 int main(.)
286 {
287     int x, i;
288     int bin[20];
289
290     cout << "pls, enter data (int) : ";
291     cin >> x;
292     //dec2bin(x, bin, 0, i);
293     dec2hex(x, bin, 0, i);
294
295     for(int j = i-1; j >= 0; j--) {
296         switch(bin[j]) {
297             case 10: cout << 'A'; break;
298             case 11: cout << 'B'; break;
299             case 12: cout << 'C'; break;
300             case 13: cout << 'D'; break;
301             case 14: cout << 'E'; break;
302             case 15: cout << 'F'; break;
303             default:
304                 cout << bin[j];
305         }
306     }
307
308     return 0;
```

โปรแกรมแปลงฐาน 2 เป็นฐาน 10

```
336 #include <iostream>
337 #include <string.h>
338 using namespace std;
339
340 int main()
341 {
342     ....char bin[20];
343     ....//int bin;
344     ....cin >> bin;
345     ....
346     ....int dec=0;
347     ....int p=1;
348     ....for(int i=strlen(bin)-1; i>=0; i--) {
349     ....    ....int d;
350     ....    ....switch(bin[i])
351     ....    ....{
352     ....    ....    ....case '0':
353     ....    ....    ....    ....d = 0;
354     ....    ....    ....    ....break;
355     ....    ....    ....case '1':
356     ....    ....    ....    ....d = 1;
357     ....    ....    ....    ....break;
358     ....    ....    ....}
359     ....    ....dec = dec + p*d;
360     ....    ....p *= 2;
361     ....    ....}
362     ....cout << dec;
363 }
```

Recursive Function vs Recursive Mathematic Function

Factorial

$$n! = n * (n - 1) * (n - 2) * (n - 3) * \dots * 3 * 2 * 1$$

$$0! = 1$$

$$1! = 1$$

$$2! = 2 \times 1 = 2$$

$$3! = 3 \times 2 \times 1 = 6$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

Factorial Formula

Factorial formula by **Product**

$$n! = \prod_{k=1}^n k$$

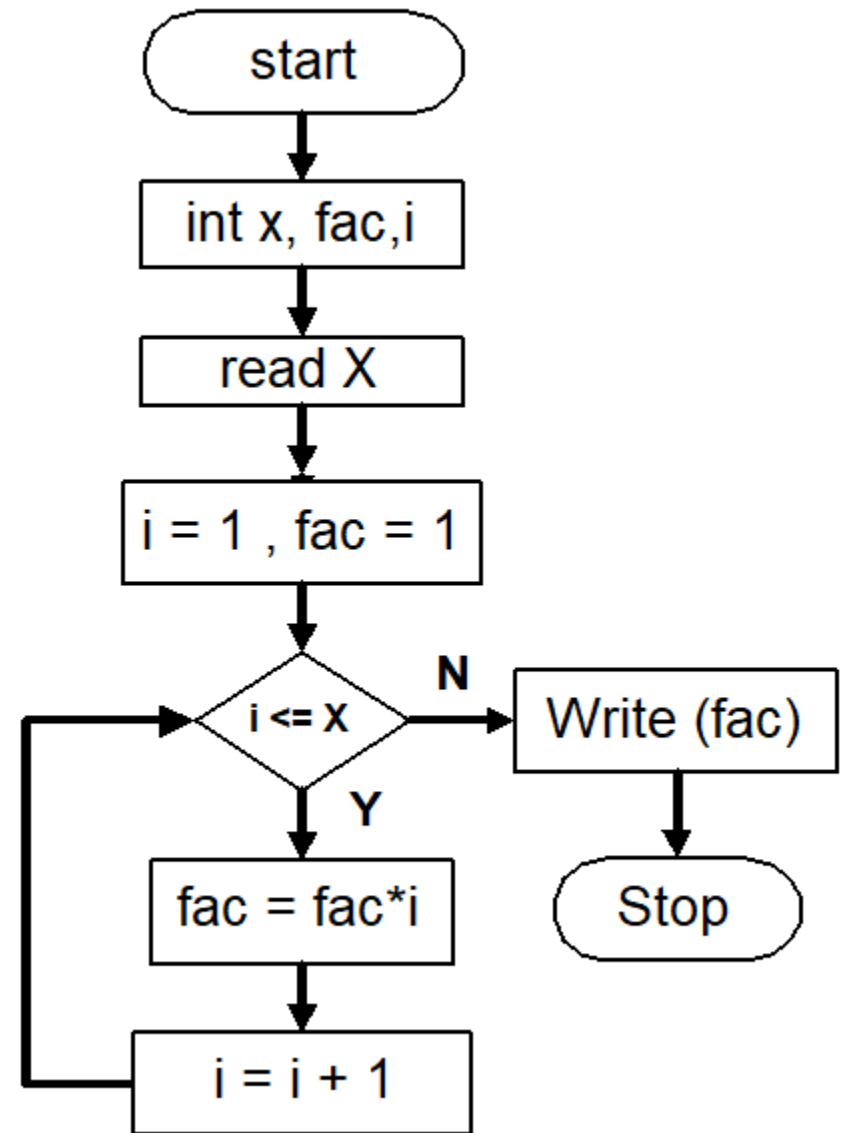
Factorial Formula by **Recurrence Relation**

$$n! = \begin{cases} 1 & \text{if } n = 0, \\ (n - 1)! * n & \text{if } n > 0 \end{cases}$$



$$n! = \prod_{k=1}^n k$$

```
1
2 #include <iostream>
3 using namespace std;
4
5 int main( )
6 {
7
8     int x , fac , i ;
9     cout << "pls, enter data (int) : ";
10    cin >> x ;
11    fac = 1;
12    for( i = 1 ; i <= x ; i++ )
13    {
14        fac = fac * i;
15    }
16    cout << fac << endl;
17
18 }
19
```



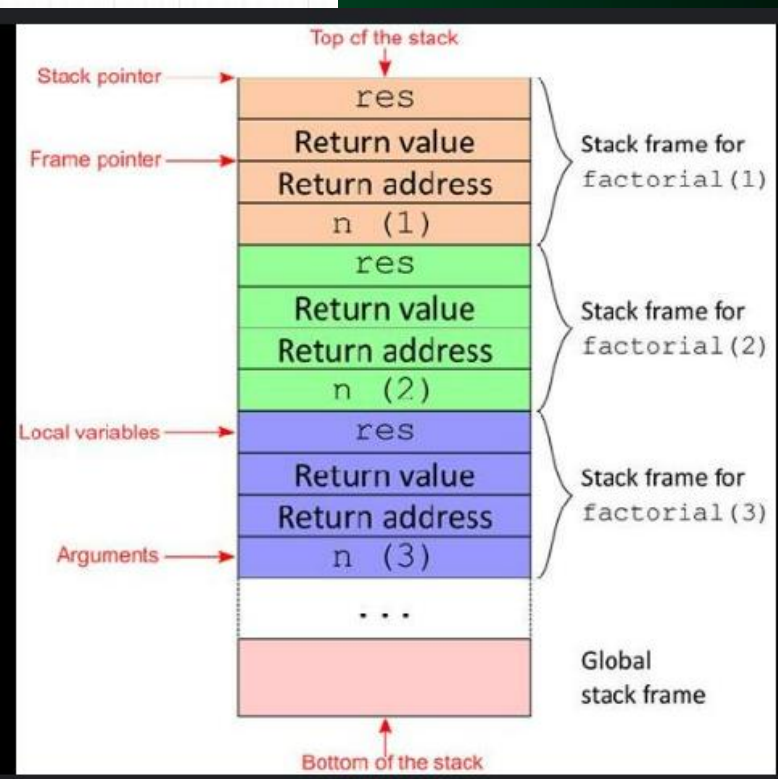
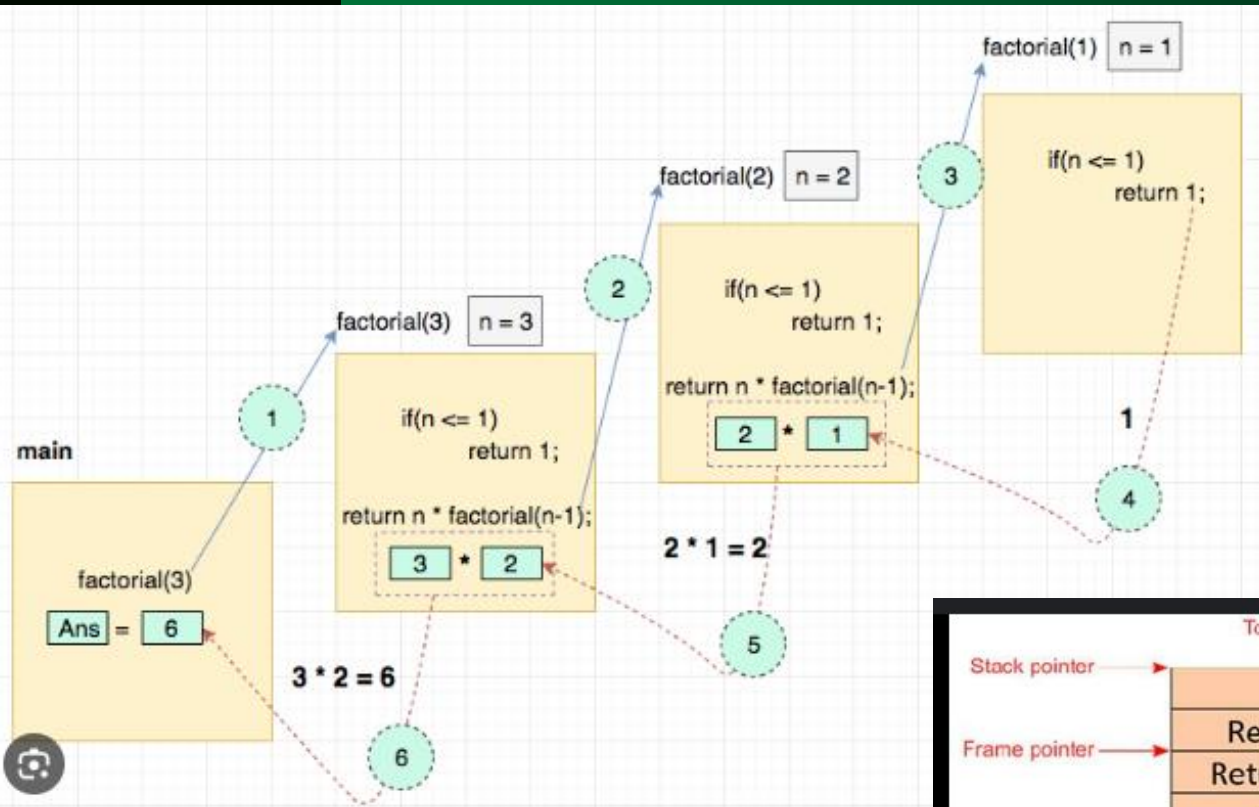
Factorial Formula

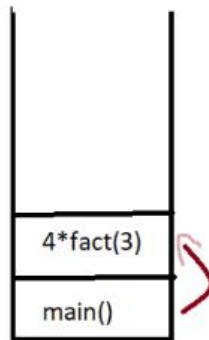
Factorial formula by **Product**

$$n! = \prod_{k=1}^n k$$

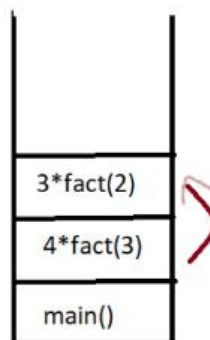
Factorial Formula by **Recurrence Relation**

$$n! = \begin{cases} 1 & \text{if } n = 0, \\ (n-1)! * n & \text{if } n > 0 \end{cases}$$

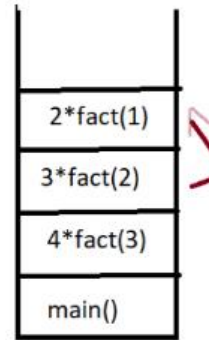




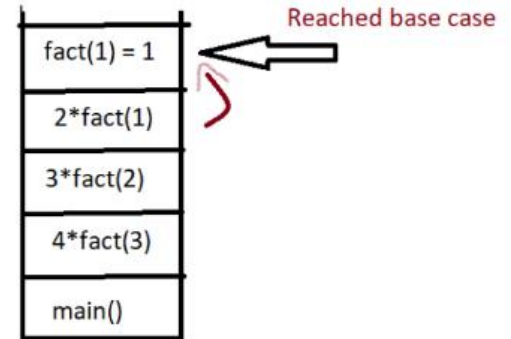
1st call



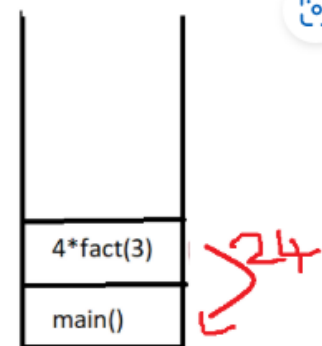
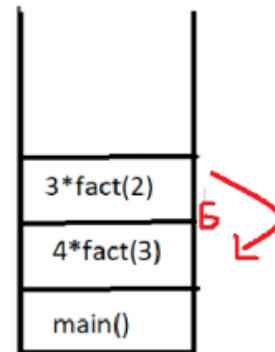
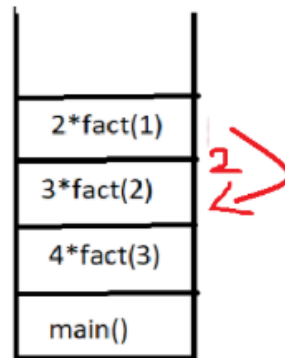
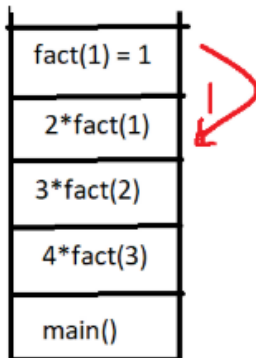
2nd call



3rd call



4th call





HOMEWORK

จงแปลงโปรแกรมต่อไปนี้ให้เป็น recursive function

```
1
2  #include <iostream>
3  using namespace std;
4
5
6  int main()
7  {
8      int sum =0;
9      int y = 50;
10     do {
11         sum = sum+y;
12         y = y - 10;
13     } while( y > 10);
14     cout << "sum = " << sum;
15     return 0;
16 }
17
```



ข้อ 6 กำหนดฟังก์ชัน $CV(N)$ เมื่อ $N \geq 1$

$$CV(1) = 2$$

$$CV(2) = 5$$

$$CV(3) = CV(1) * 2 + 1 = 5$$

$$CV(4) = CV(2) * 3 - 5 = 10$$

$$CV(5) = CV(3) * 2 + 1 = 11$$

$$CV(6) = CV(4) * 3 - 5 = 25$$

จงเขียนโปรแกรม C++ โดยใช้ recursive subprogram เพื่อทำการคำนวณหาค่า $CV(N)$ เมื่อ N คือ INPUT DATA

ข้อ 5 พิจารณาผลบวกของข้อมูลต่อไปนี้

$$(1/10) + (2/10) + (3/10) + (4/10) + \dots + (n/10)$$

สมมติว่า n เป็นเลขจำนวนเต็มบวกใดๆ

ถ้า $n=1$ ผลคือ $1/10$

$n=2$ ผลคือ $(1/10) + (2/10)$

$n=3$ ผลคือ $(1/10) + (2/10) + (3/10)$

จงเขียนโปรแกรมสมบูรณ์ที่มีการเรียกใช้ฟังก์ชันชื่อ SUM ที่มีการส่งผ่านข้อมูล n ใดๆ และส่งผ่านค่ากลับไปยังจุดเรียกใช้



ข้อ 7 จงเขียนโปรแกรมสมบูรณที่มีการเรียกใช้ฟังก์ชันเรียกตัวเองเพื่อทำงานดังนี้

$$(1/1!) + (1 / 2!) + (1/3!) + \dots + (1/n!)$$